Marginalia

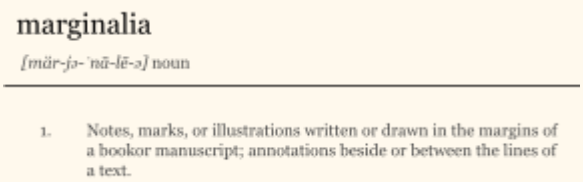
Arnald Paguio

1 INTRODUCTION

A few months ago, I was reading a book on my phone and noticed myself constantly pasting excerpts into a Claude chat to help explain parts of the book. At multiple points, I wished the AI was just built into the reading app. I searched online for e-readers that fit the bill, but it quickly became clear I'd have to build it myself. Hence, **Marginalia** was born.

Beyond wanting to build a cool app, Marginalia also represents my discontent with the current AI paradigm. Most AI tools are built around one prerogative: go faster. AI's value proposition has become synonymous with productivity — but I think that's a narrow, utilitarian view of how it can benefit humanity. I wanted to explore AI's value beyond labor delegation, and reading felt like the perfect space for that exploration. Hence, Marginalia's ultimate goal is the **curation of an experience**, not the optimization of a task. Marginalia should feel like walking into a library whose librarian is there to read with you, learn your preferences, and guide you on your reading journey.

What is Marginalia?


marginalia

[mär-jə-ˈnā-lē-ə] noun

1. Notes, marks, or illustrations written or drawn in the margins of a bookor manuscript; annotations beside or between the lines of a text.

Marginalia is a local-first, open-source desktop EPUB reader with an AI reading companion built into the act of reading. It is built with Tauri v2 (Rust + React), using SQLite as the local database. Much of the backend was inspired by another open-source e-book reader, **Readest**. I transplanted the bare minimum components from Readest to have a functioning e-reader, but everything else was built from scratch. Marginalia is built on a Bring-Your-Own-Key model (BYOK), so any user has complete control over how much cost they put into AI-use. Support is currently limited to MacOS and Anthropic models, but this will be addressed in the future!

The Blind Model

Marginalia is designed around what I call the Blind Model. By default, the AI has no access to the book. It works only from the passage the reader has selected, the question they've asked, and a lightweight reader profile built up over time. It cannot see the surrounding chapters, scan ahead, or draw on the full text unless it explicitly asks for more.

This is intentional. Most AI reading tools (i.e. NotebookLM) load entire chapters into context and let the model answer from a position of total knowledge. The result tends to be comprehensive, confident responses that do a lot of the reading for you. Marginalia resists that. Keeping the model blind forces it to reason carefully from what's in front of it — and when it genuinely needs more context, it fetches it through explicit tool calls that are visible and auditable. The desired effect is that responses stay focused and Socratic rather than encyclopedic. The model reads with you, not ahead of you. This behavior is reinforced through the design of the model's harness and through explicit prompting.

2 FEATURES

Threads

Threads are basically chats inside a book that are persistent and stored locally. You can start a new thread for a different part of the book, pick up an old one mid-chapter, or run several threads in parallel for different lines of inquiry. Nothing is ephemeral. The typical flow would be while reading, you'd attach a selected passage from the book to a thread (much like how you can attach files or lines of code in AI-powered IDEs), and then ask the model about that selected passage. Along with the selected passage, the CFI-index (the location of the

passage in the EPUB) is also passed to the model. This gives the model a specific location anchor in the book from which it can use to guide its tool-use. Behind the scenes, the text leading up to the selected passage is also given to the model to help give the model a bit more context. Early iterations of Marginalia had a radius of text surrounding the selected passage passed by default, but I found this made the model more prone to spoiling sections beyond the selected passage.

Smart Scan

Smart Scan is a user-triggered workflow that runs when you first open a book and want the AI to be genuinely useful beyond the immediate passage. When invoked, it walks the book section by section and generates a short summary for each one, then assembles those summaries into a book map — a structured outline of what's where. That map gets injected into the model's system prompt at the start of every thread. The payoff is retrieval: when you ask a question that requires finding something elsewhere in the book, the model can consult the map to identify which section is likely relevant, then fetch it. The book map contains CFI-index ranges as well and contains a marker if the section has already been read or is ahead of the user's location in the book. Without Smart Scan, the model is completely blind to anything outside the selected passage. I generally recommend to use the Smart Scan for all books–the upfront cost of Smart Scanning a book is made up for better retrieval and response accuracy. I don't have exact figures, but a typical Smart Scan for a 300-400 page book costs on average 10 cents using Claude Haiku 4.5. I'd love to explore embedding an entire book, but due to time and budget constraints, I cached this for later iterations.

Tools

Tools. The model has access to two tools. get_context fetches a window of text around the current passage — useful for nearby evidence that didn't make it into the selection. get_section fetches a full section by name from the book map — the primary retrieval tool when the answer lives somewhere else in the book. Both are visible and auditable: you can see when the model calls them and what it pulled. Tool calls count against the three-turn agentic limit, after which the model must respond with what it has.

Memory

Marginalia remembers across sessions at two levels. Per-book memories track the thread of your reading experience with a specific book — recurring themes you've noticed, questions you keep returning to, interpretations you've built up. Global memories capture patterns across books: the kinds of questions you tend to ask, authors or ideas that keep coming up, how you like to engage with difficult passages. Both live locally in SQLite and are injected into the system prompt at the start of each new thread.

Memory is populated through compaction, which runs at two points. The first is a mid-thread flush: once a thread accumulates five exchanges, Marginalia quietly runs a background pass over the conversation using Claude Haiku, extracts any high-confidence observations, and writes them to the memory store. This happens once per thread and is non-blocking. The second pass runs on archive: when you manually archive a thread, a more thorough extraction runs over the full conversation, pulling up to six memory items across book-level and global scopes. Before saving anything, the system checks against existing memories for near-duplicates — items that already exist get reinforced rather than added again, keeping the memory store from inflating over time.

Retrieval works differently at each level. Book memories are fetched by recency — the four most recently reinforced items for the current book are included. Global memories use vector search: each memory item is embedded on save using BGE-Small-EN-v1.5 (quantized), a lightweight model that runs entirely on-device via the **fastembed** Rust crate. No API call, no network round-trip. When a new thread starts, the user's opening message is embedded the same way, and the top four global items by cosine similarity are selected. If the embedding lookup fails for any reason, retrieval falls back to recency. The book text itself (sections, highlights, thread history) is not embedded; only the distilled memory items are. Hybrid search (combining vector similarity with keyword scoring) is a natural next step and will be explored in the future.

3 EVALUATION STRATEGY

I focused the evaluation of Marginalia around the **reliability and accuracy of its context-retrieval capabilities**. Multiple single-thread interactions were simulated across three public-domain books: **(1) Pride and Prejudice** (Jane Austen), **(2) Frankenstein** (Mary Shelley), and **(3) Walden, and On The Duty Of Civil Disobedience** (Henry David Thoreau). The goal was to test whether Marginalia could consistently locate relevant evidence, ground its answers in the text, and maintain quality across different writing styles and narrative structures. Claude Haiku 4.5 was the model used during these evaluations. All three books were sourced from Project Gutenberg.

Prompt Categories

Prompts were grouped by the type of evidence they required and how far that evidence was likely to sit from the active passage. Each category represents a distinct retrieval challenge, from simple local lookups to broader synthesis, with grounding and evidence quality prioritized over stylistic fluency.

Prompt Category	Description
Passage-local	Further context isn't needed in the selected excerpt
Nearby-context	Query needs text immediately before/after the excerpt
Book-level thematic	Prompt asks for broader themes/patterns across the book
Cross-section	Prompt required connecting evidence from separate sections

Retrieval Strategies

Three configurations were compared to isolate the contribution of tool use and structured indexing. Prompts were held constant across all conditions so that score differences reflect retrieval strategy rather than prompt variation.

Retrieval Strategy	Description
No tools	Model can only use the selected passage alone and the text leading up to the selected passage
get_context only	Model can fetch text immediately before or after the selection
Smart Scan + All tools	Model has full tool access and section maps/summaries

Metrics

Each model response was then tested against **four metrics**, with each being scored by a separate LLM as a judge. The judge LLM was given a rubric for each metric on how to score the model's responses to the prompt. Each score ranges from 1-5, with 5 being the highest. A score of 3 would be considered sufficient. The model used for judging was also Claude Haiku 4.5.

Metric	Description
Faithfulness	Does the answer stay true to what the source text and retrieved evidence actually say?
Chunk Relevance	Did retrieval pull the right evidence for the question?
Answer Completeness	Does the response fully address what was asked
Claim Precision	Are claims specific, or vague/over-broad?

In total, 130 **(15 prompts x 3 Retrieval Tiers x 3 Books)** responses were generated and evaluated. Sadly, I wasn't able to track the actual costs per response, but based on the Anthropic developer console, 130 tests came out to roughly 50 cents.

4 EVALUATION RESULTS

A small caveat before proceeding: I'm mostly interested in the Smart Scan + all tools results, because that is the normal state of the app. The other two conditions are useful as ablations to verify the harness meaningfully improves performance.

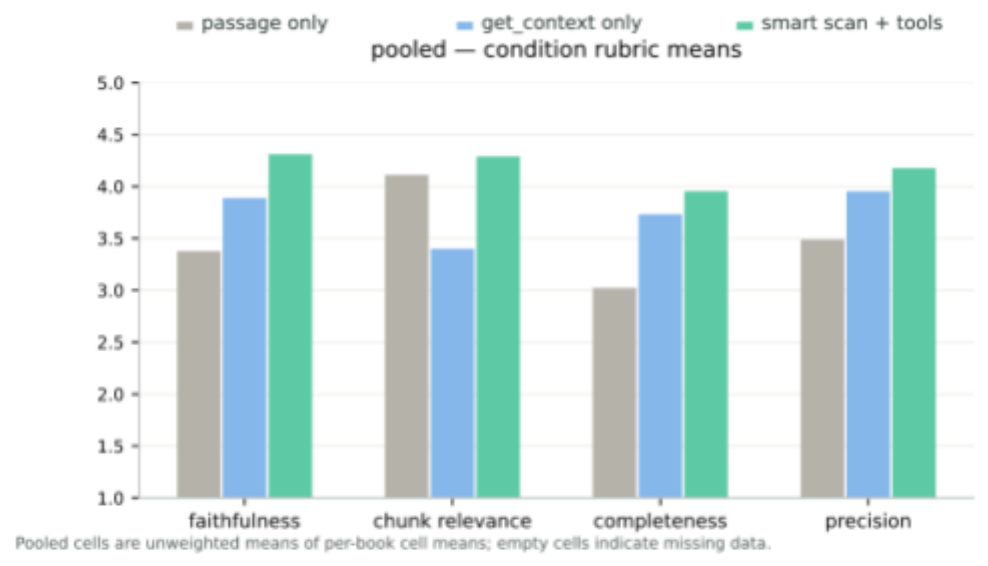


Figure 1: Mean-pooled scores (across all three books)

In the mean-pooled scores chart (Figure 1), Smart Scan + all tools is the strongest configuration overall, averaging about 4.19 across all books and metrics compared with about 3.76 for get_context only and 3.46 for no tools. That matches the intuition behind the system: local context helps, but the app becomes much more reliable when the model can orient itself with section-level summaries and the book map before choosing what to read.

The average score hides a lot of variance, though. By breaking the scores down by prompt category **(Figure 2)**, you'll see where the system actually struggles. Passage-local prompts are strong almost everywhere, which is not surprising: if the answer is already inside the selected excerpt, the model does not need much retrieval help. Nearby-context prompts also do well once tools are available, because the missing evidence is usually just before or after the anchor. The hard cases are cross-section and book-level thematic questions, where the assistant has to leave the immediate passage and find a separate part of the book. That is where get_context-only starts to look structurally limited: it can widen the window around the current passage, but it cannot really choose a distant section on its own.

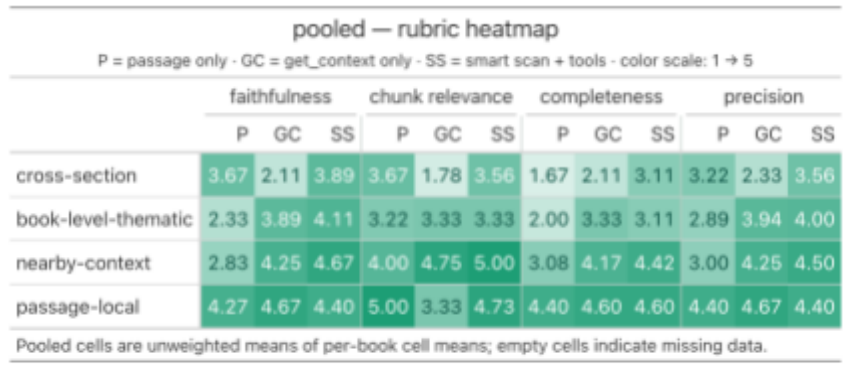


Figure 2: Prompt Category x Retrieval Strategy Heat Map

Failure Modes

Even with the complete tool-set and smart-scan assistance, Marginalia exhibited notable failure modes. The main ones I observed were: (1) stopping after partial evidence, (2) choosing a section that was thematically nearby but not decisive, and (3) turning incomplete evidence into a more confident answer than the text justified.

One Walden example shows this clearly:

> ***Question***: "He is arguing about clothing and shelter as defenses against cold—where does he revisit ice, water, and bathing as a different kind of bodily discipline?"
>
> ***Selected Passage***: "a cold world; and to cold, no less physical than social, we refer directly a great part of our ails…"

Smart Scan retrieved a large amount of relevant-looking Economy material, but the answer leaned on an accidental cold-water episode rather than a clearly deliberate bodily discipline. The problem was not that nothing was retrieved, but rather the retrieved evidence sounded adjacent enough that the model treated it as sufficient.

Frankenstein produced a similar cross-section miss.

> ***Question***: "Victor is demanding official pursuit of his enemy here—where earlier did the creature first ask Victor for a mate, framing obligation vs. threat?"
>
> ***Selected Passage***: Victor telling the magistrate, "I do not doubt that he hovers near the spot which I inhabit…" [and insisting that his enemy should be hunted and punished.]

The answer needed the earlier Alpine confrontation where the creature demands a female companion. In the failed Smart Scan run, the system did not complete that jump and failed to locate or quote the mate-demand scene and gave an incomplete answer. This is the main remaining retrieval challenge for Marginalia: cross-section questions require not just a plausible section, which implies the model requires more precise searching tools.

Overall, the evaluation supports Smart Scan as the right default. It improves the assistant most in the places where questions move beyond the highlighted passage. The get_context tool is still valuable, but mainly as a precision tool once the anchor is known. The next retrieval fixes are more concrete: **explore embedding the entire book so cross-section lookup is not limited to section summaries**, and relax the current three agentic-turn limit so the model has more room to search, inspect, and then answer. For evidence overreach, the likely fix is stronger prompting: make the model explicitly separate what was retrieved from what is only plausible, and push it to say when the evidence is not enough instead of smoothing over the gap.

5 SAFETY AND ETHICAL CONCERNS

Due to time constraints, no formal safety testing was conducted on Marginalia. However, several areas do warrant serious attention.

The most significant concern is **content-conditioned jailbreaking**. Marginalia feeds book passages directly into the model's context, and books can contain graphic violence, explicit content, or extremist material that a user might legitimately be reading. This creates a passive attack surface: harmful content in context can lower the model's refusal threshold, and a determined user could exploit the "literary discussion" framing to extract outputs the model would otherwise decline. Mitigations worth exploring include content classification at the passage level before context injection, and system prompt hardening that maintains behavioral constraints regardless of surrounding content.

A second concern is **prompt injection via EPUB content**. Since arbitrary text from the book enters the model's context, a crafted document could embed text designed to look like system instructions. This is a known vulnerability in any system that mixes untrusted data with prompt context, and Marginalia currently has no sanitization layer between the parsed EPUB text and the model.

Finally, the **BYOK model** means API keys are stored locally and managed by the user. The current implementation should be audited to ensure keys are not accessible to other processes or inadvertently exposed through logs.

5 CONCLUSION

The Harness Matters as much as the Model

Building Marginalia taught me that the quality of an AI system has less to do with the model you're using and more to do with everything around it– how context is assembled, what the model is and isn't shown, how retrieval is triggered, and what constraints are baked into the system prompt. I came into this project thinking the interesting problem was the AI. I left it thinking the interesting problem is the engineering that shapes how the AI behaves. Smarter models don't automatically produce better systems. A more capable model handed a poorly designed harness will fail in more sophisticated ways! This was also humbling in a different way: failure modes are incredibly hard to anticipate in advance. Many of the issues I encountered— the model not reaching for tools when it should, responses that were technically correct but contextually wrong, retrieval that fired too eagerly or not at all — weren't obvious until I was watching real behavior on real books. You can reason about a system carefully and still be surprised by it. More testing, on more books, with more adversarial prompts, would have caught things I didn't.

It Actually Works!

I've been using Marginalia as my primary e-reader for the past several weeks, and I genuinely prefer it to the alternatives. The ability to highlight a passage and immediately ask a question about it turns out to be a meaningful quality-of-life improvement. It changes how I engage with difficult texts. That, more than any benchmark result, tells me the core idea is sound.

What Comes Next

There's plenty left to improve. On the technical side, I'd like to explore chunking and embedding entire books to enable richer semantic retrieval, and invest in more rigorous cost estimation — my full evaluation suite of 130 runs cost roughly fifty cents, which suggests the per-user economics are favorable, but that needs proper modeling across usage patterns. Support for other model providers and platforms beyond MacOS is an obvious gap. And the UI, while functional, has accumulated enough rough edges that a focused design pass is overdue.

Most importantly: more testing. Safety testing, adversarial testing, testing on a wider range of books and reading behaviors. Marginalia works, but working and being robust are different things, and right now it's much more the former than the latter.